

Databricks and Spark Optimization

Follow the data from 1 TB of files to the final Delta table

A practical guide for a Lead Data Engineer working with retail forecasting data products

You already build pipelines. This guide opens the parts that are normally hidden: the bytes inside a Parquet file, the table snapshot in Delta, the batches a reader hands to the execution engine, the rows that move during a shuffle, and the files a writer creates. The aim is to make a slow job understandable before changing its configuration.

Follow the chapters in order the first time. Keep the sample row identifiers beside you. Later, use the diagnostic chapters and the two supplied query-profile photographs as a reference. SQL and Python examples are deliberately small enough to connect back to the rows.

All retail rows, file sizes, execution distributions and benchmark results in the teaching examples are illustrative. The Parquet metadata lab uses a real locally generated file. The screenshot chapter distinguishes visible evidence from hypotheses. Platform guidance was checked against official documentation on 23 September 2026; inspect the actual runtime and compute configuration before applying a setting.

The main conclusion is simple: reduce unnecessary data first, preserve the correct business grain, then make the remaining work run efficiently. A faster engine is useful, but it cannot decide whether the join or the feature definition is correct.

Reading map

Chapters	Learning path
1 and 2	Start with the sample rows and distinguish files from execution partitions
3 to 5	Open Parquet and follow pruning through real footer statistics
6	Connect the reader to the execution engine
7 to 9	Understand Delta commits, versions, failures and row corrections
10 and 11	Follow jobs, stages, tasks and actual shuffle movement
12 to 14	Choose joins and understand skew, AQE and repartitioning
15 to 17	Build correct rolling features and publish safe incremental or backfill results
18 to 20	Diagnose every layer and review settings with evidence
21	Compare standard Spark SQL execution with Photon
22 to 24	Walk through both supplied query-profile photographs
25	Work through a complete illustrative tuning exercise
26 and 27	Practise twenty interview questions and a natural spoken explanation
28 and 29	Use the one-page checklist and concept map
30	Produce a fourteen-minute video from timed scenes and exact narration
31	Consult official references and the requirement coverage map

The Word navigation pane lists every chapter and interview question. The PDF follows the same chapter order. Read the storage chapters before the settings chapter; otherwise, terms such as partition size and file size can easily be confused.

1 Our retail data and the result we want

We will use a sales-history Delta table on ADLS Gen2. Its active Parquet files occupy 1 TB in the large example. The source grain is a sales transaction line. Our forecasting output grain is one row per item, location and business date. Grain means exactly what one row represents.

The miniature table has twelve transaction lines. A and B are item IDs; S1 and S2 are stores. R01 to R12 are teaching labels for individual source rows. They help us follow a row across files and workers.

Row	Date	Item	Store	Qty
R01	2026-09-01	A	S1	2
R02	2026-09-01	A	S1	3
R03	2026-09-01	B	S1	4
R04	2026-09-01	B	S1	1
R05	2026-09-02	A	S1	5
R06	2026-09-02	A	S2	2
R07	2026-09-02	B	S1	1
R08	2026-09-02	B	S2	1
R09	2026-09-03	A	S1	6
R10	2026-09-03	A	S2	3
R11	2026-09-03	B	S1	2
R12	2026-09-03	B	S2	5

For A at S1 on 1 September, R01 and R02 must become one daily row with quantity 5. For B at S1 on 1 September, R03 and R04 become quantity 5. The complete daily result has ten rows and total quantity 35. Aggregation changes the grain but must preserve this sum.

Date	Item	Store	Daily quantity
2026-09-01	A	S1	5
2026-09-01	B	S1	5
2026-09-02	A	S1	5
2026-09-02	A	S2	2
2026-09-02	B	S1	1
2026-09-02	B	S2	1
2026-09-03	A	S1	6
2026-09-03	A	S2	3
2026-09-03	B	S1	2
2026-09-03	B	S2	5

We join a unique item lookup, a unique store lookup and a promotion mapping that is unique at its intended effective-date grain. Then we calculate daily quantity and historical rolling features. A nightly run reads a configurable range. A correction may require recalculating later dates because a rolling feature depends on earlier days.

The twelve rows are not representative performance data. They are a microscope. When we scale to 1 TB, the meaning of the operations stays the same, but there are more files, more input partitions, more tasks and more opportunities for uneven work.

A natural way to explain this is: “One of the main areas I look after is preparing reusable data for forecasting. I start by making the grain clear. Here the source is transaction level, and the output is item, store and day. That helps me check both the joins and the final totals.”

2 What 1 TB means on storage

In this guide, 1 TB means 1,000,000,000,000 bytes of active compressed Parquet data. That is different from 1 TiB, which is 1,099,511,627,776 bytes. It is also different from the size of the same values after decoding in memory. Never estimate executor memory by assuming compressed file bytes expand by one fixed ratio.

Average compressed file size	Approximate files for 1 TB	What changes
10 MB	100000	Many objects to discover and open
128 MB	7813	More independent pieces
256 MB	3907	Fewer opens with useful parallelism
512 MB	1954	Fewer objects and larger reads
1 GB	1000	Larger pieces and potentially coarser skipping

These are decimal sizes, rounded up. They are illustrations, not a recommended target. A very selective workload may benefit from a different layout than a nightly scan of most of the table. File count alone does not tell us scan task count.

An ADLS location can contain the following objects. Directory names are logical path prefixes; Delta membership comes from the transaction log, not from listing everything ending in .parquet.

```
abfss://retail@account.dfs.core.windows.net/sales/
  part-F0.parquet      R01 to R04
  part-F1.parquet      R05 to R08
  part-F2.parquet      R09 to R12
  _delta_log/
    00000000000000000000000000000000.json
    00000000000000000000000000000001.json
    ... later checkpoint files ...
```

Term	Meaning in this example
File	One physical Parquet object such as F0
Row group	A horizontal group of rows within a file
Storage partition	A table layout division such as sale_date=2026-09-01
Spark partition	One piece of a distributed dataset at a particular stage
Task	One attempt to process a Spark partition for a stage
Executor	A worker process that can run multiple tasks
Task slot	Capacity to run a task concurrently, subject to resource needs

F0 can hold several row groups. A scan may group small files into one input partition or split a larger Parquet file into supported scan ranges. A task is not permanently attached to a particular file or executor. An executor may process many partitions over the life of the application.

Observe active size and file count with DESCRIBE DETAIL, then compare scan files, bytes and input tasks in the executed plan. Do not expect those numbers to be equal. [1, 2]

3 Open a Parquet file and see the columns

Parquet stores values by column within a group of rows. It does not store all dates for the entire table in one global date file. Inside each row group, each column has its own column chunk. A column chunk contains encoded pages. A page is a smaller unit of encoded data and compression. [1, 3]

Take R01 to R04. Their row view is easy for a person to read. The storage view brings values from the same column together.

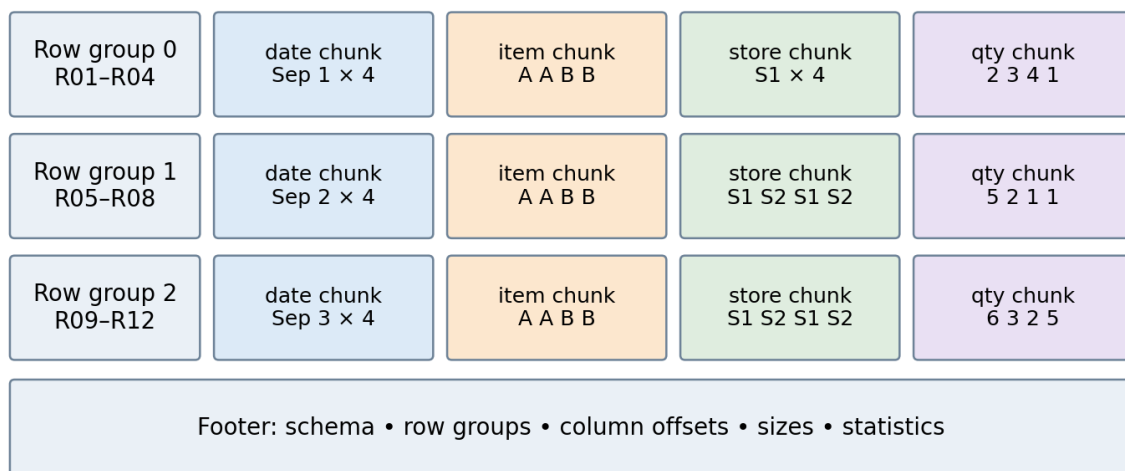
Column chunk in row group 0	Values in source row order
sale_date	Sep 1, Sep 1, Sep 1, Sep 1
item	A, A, B, B

Column chunk in row group 0	Values in source row order
store	S1, S1, S1, S1
qty	2, 3, 4, 1

The logical row can still be reconstructed: position 0 in each column belongs to R01. For optional and nested data, Parquet also stores structural information so that nulls and nesting can be reconstructed. A file is not a CSV transposed into readable text; it is binary data with encoded pages and metadata.

For a deliberately tiny conceptual page example, split the quantity chunk into page 0 holding [2, 3] and page 1 holding [4, 1]. Their ranges are [2, 3] and [1, 4]. A `qty > 3` predicate can exclude page 0 if suitable page statistics/indexes and reader support are present, but page 1 still needs filtering because its value 1 does not qualify. The actual lab writer chooses its own page boundaries; these two tiny pages illustrate the hierarchy rather than describe the lab's physical page count.

One Parquet file



Parquet column chunks inside row groups

Why does this compress well? A store column containing S1 repeatedly can use a small dictionary and compact codes. Consecutive repetitions can be encoded efficiently. Numeric columns can use encodings appropriate to their values, followed by a compression codec such as Snappy or Zstandard. Encoding represents values compactly; compression reduces the byte representation further. The actual choices depend on the writer and data. [4]

If our query needs only date, item and quantity from a forty-column table, the reader can avoid loading the other column chunks unless they are needed for filters or expressions. Reading three out of forty columns does not guarantee exactly 3/40 of the bytes: columns differ in width and compressibility, and metadata still has to be read.

A transactional row-oriented database can be more suitable when an application repeatedly retrieves or updates a single customer or order record and needs low-latency indexed operations. Parquet is well suited to analytical scans over many rows and relatively few columns.

Use this explanation in an interview: "Parquet helps us avoid reading columns that the query does not need. Similar values are also stored together within each row group, so they often compress well. I still check bytes read, because a narrow SELECT does not tell me how selective the file layout is."

4 The footer and statistics as actual metadata

The footer is a map at the end of an ordinary Parquet file. It describes the schema, row groups and column chunks, including offsets, sizes and available statistics. It does not contain all the data rows. In the usual unencrypted layout, the file begins with PAR1 and ends with serialized metadata, a four-byte metadata length, and PAR1. Encrypted variants differ. [1, 3]

The reader can first fetch the tail, decode the metadata length, and locate the footer. It then knows where the needed column chunks live. Object storage supports range reads, although the implementation may cache metadata or combine requests; one footer does not imply one network request for every field.

For this lab, the twelve sample rows were written into one Parquet file with three row groups of four rows. PyArrow is used only to expose real Parquet structure locally; it is not a claim that Spark or Photon produces byte-identical files.

```
{
  "file": "retail_sample.parquet",
  "created_by": "parquet-cpp-arrow version 25.0.1",
  "file_size_bytes": 3173,
  "num_rows": 12,
  "num_row_groups": 3,
  "schema": [
    "row_id: string",
    "sale_date: DATE",
    "item: string",
    "store: string",
    "qty: INT64"
  ],
  "footer_start_byte": 1245,
  "footer_length_bytes": 1920,
  "trailer": {
    "metadata_length_little_endian_hex": "80070000",
    "magic": "PAR1"
  }
}
```

The metadata below is a readable extraction. On disk it is serialized binary metadata, not this JSON-shaped display. Offset and size values come from the generated file. Dates are displayed as dates even though Parquet DATE uses an underlying integer representation.

Group	Column	Min	Max	Nulls	Data offset	Compressed bytes
0	sale_date	2026-09-01	2026-09-01	0	110	78
0	item	A	B	0	194	66
0	qty	1	4	0	340	117
1	sale_date	2026-09-02	2026-09-02	0	521	78
1	item	A	B	0	605	66
1	qty	1	5	0	752	112
2	sale_date	2026-09-03	2026-09-03	0	934	78
2	item	A	B	0	1018	66
2	qty	2	6	0	1170	117

Offsets are zero-based byte positions in this generated file. A data-page offset is not necessarily the start of the column chunk: a dictionary page may precede it. Compressed size covers the column chunk, not only one data page.

A minimum and maximum give a safe range for excluding a row group. They do not tell us which exact values occur inside the range. If a group has item minimum A and maximum Z, an item M predicate cannot be ruled out, even if M is absent. Null counts may help some predicates. Missing statistics mean the reader cannot rely on that statistic to skip work.

Before reading, the file has twelve rows and three row groups. With the predicate `sale_date >= 2026-09-02` and `sale_date < 2026-09-03`, row groups 0 and 2 can be excluded using these date ranges. Row group 1 remains, and ordinary predicate evaluation ensures correct results. This is the small equivalent of eliminating irrelevant history before decoding millions of values.

5 Follow a selective query through the reader

Consider a forty-column sales table. This query requests three columns and a seven-day interval.

```
SELECT sale_date, item, qty
FROM retail.sales
WHERE sale_date >= DATE '2026-09-01'
AND sale_date < DATE '2026-09-08';
```

The engine needs `sale_date` for both output and filtering. It does not need store for this particular query. A different query that filters on store while selecting only the other three columns still needs store to evaluate that filter.

Step	Evidence used	Example of work avoided
Partition pruning	Table partition values	Exclude August directory partitions
Delta file skipping	Active-file statistics	Exclude a file whose maximum date is August 31
Column pruning	Required scan schema	Do not decode the other 37 columns
Row-group pruning	Parquet column statistics	Skip a group entirely outside September 1 to 7
Optional page skipping	Available indexes and reader support	Skip eligible pages within a surviving chunk
Row filtering	Actual decoded values	Remove remaining nonmatching rows

Predicate pushdown means passing a filter closer to the data source or reader so it can reduce work. It is not a separate promise that every predicate skips files, groups and pages. A plan showing `PushedFilters` is useful evidence, but measure bytes and rows too. Optional Parquet page indexes exist; their use depends on the writer, reader, expression and execution path. [2, 5, 6]

In the miniature file, a September 2 predicate retains R05 to R08 and excludes the other groups. In a large, well-localized table, seven days may represent roughly 7/365 of one year of evenly sized history before column pruning. That is only an estimate: seasonal volumes, file overlap and row widths change it.

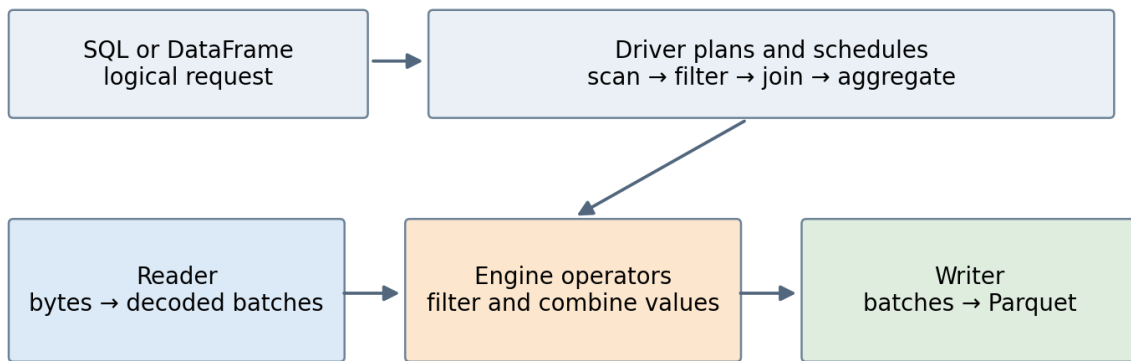
Now rearrange the miniature rows so every row group contains September 1, 2 and 3. Every group has overlapping date statistics. The same September 2 filter returns the same four rows, but cannot exclude any group based on date min/max. Selectivity in the answer is not the same as selectivity in storage reads.

If a query returns few rows but scans most files, first compare its predicates with the layout and available statistics. Adding executors may finish the avoidable scan sooner but still pays to read irrelevant data. My spoken explanation would be: “I check how much data we read to produce the result. If most of the files are irrelevant, I look at pruning and layout before changing the cluster.”

6 What Spark and the reader each do

The reader, query engine and writer are cooperating components. The reader knows how to obtain and decode storage data. The engine applies operators such as filters, joins and aggregates. The writer encodes results into files. Delta adds the table transaction around those files.

A Python `DataFrame` call constructs a query description. It does not mean Python loops over every sales row. Spark resolves names and types, creates a logical plan, applies optimizations and chooses physical operators. The driver coordinates planning and scheduling. Executor tasks perform the distributed data work. [2, 7]



Executor tasks process partitions; Delta publishes the committed file set

Reader engine and writer responsibilities

For a standard Spark SQL scan, a supported vectorized Parquet reader can decode batches of column values. Other parts of the JVM execution path may use internal rows and generated code. Do not describe all non-Photon Spark work as naïve interpreted processing of one Python row at a time. A native Photon path uses its own supported scan and execution operators.

A task reading the miniature September 2 row group obtains R05 to R08. The filter checks the date. A projection selects columns and can calculate an expression such as `qty * unit_price`. Batches pass to downstream operators. The raw compressed bytes, the decoded in-memory batch and the output rows are different representations of the same information.

Where does memory go? The reader needs buffers, the engine needs batches and working state, joins may need hash tables, and sorts or aggregates may hold intermediate values. Several concurrent tasks share executor resources. Giving an executor more cores can increase parallelism and also increase simultaneous memory demand.

A Python UDF introduces a different boundary: data may move into Python workers for custom code and return to the engine. Serialization and Python computation can become significant. Built-in expressions give the optimizer and supported native execution paths more visibility. This is why I would express a simple conditional with `when` or `CASE` rather than a custom Python function.

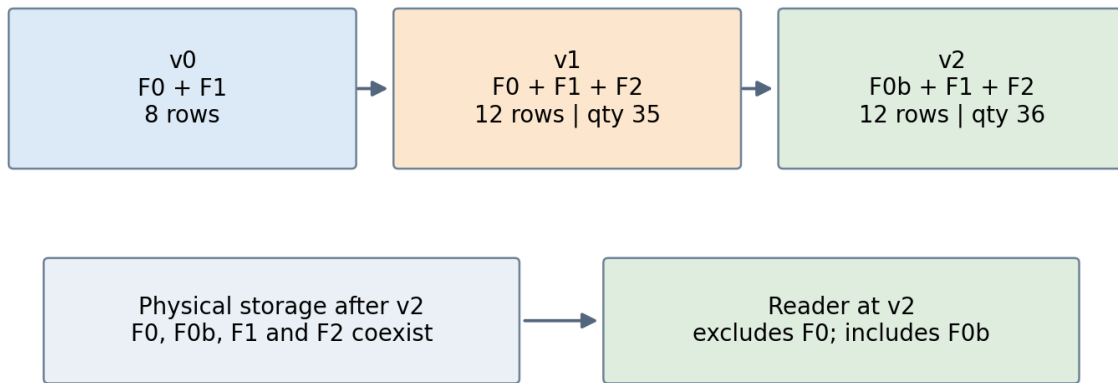
In the UI, inspect the scan schema, pushed filters, physical operators and Python evaluation nodes. If runtime is dominated by Python, changing Parquet file size may not address the main cost. If runtime is dominated by opening tiny files, rewriting the UDF may not address it either.

7 Plain Parquet and a Delta table

Plain Parquet gives us structured columnar files. A folder of those files does not by itself describe which combination constitutes one committed table version. Delta supplies that table-level metadata and transaction protocol. [8]

Imagine F0 and F1 contain September 1 and 2. A correction creates F0b. In a plain folder, a reader that indiscriminately reads F0, F1 and F0b can duplicate September 1. If another process deletes F0 at the wrong moment, a reader can fail or read an inconsistent selection. An application can add its own coordination, but the file format alone does not solve this.

A Delta reader resolves a committed snapshot: a set of active data-file references and associated metadata. An old snapshot can reference F0 and F1; a new one can reference F0b and F1. Physical files can coexist without both being logically active in the same snapshot.



Committed versions and physical file coexistence

The diagram shows a simple classic log-based example. Modern table features and commit coordination can change the physical metadata implementation, but the important contract is the committed snapshot. Do not write custom JSON files into a production `_delta_log`; use supported Delta writers.

Schema defines columns and types. Protocol and table features establish what a compatible reader or writer must understand. File statistics help avoid opening files that cannot match a predicate. A checkpoint summarizes table state so constructing a later snapshot need not replay every earlier JSON action; a checkpoint is not a copy of all table data. [8, 9]

For example, if a checkpoint represents version 100, constructing version 103 can start from that summarized state and apply commits 101, 102 and 103. The active-file set reflects adds and removes across that sequence. The actual checkpoint policy and format depend on enabled features; do not assume that every table always checkpoints on one fixed interval.

A Delta file-statistics entry for our twelve-row lab file could show numRecords 12, minimum sale_date September 1, maximum September 3, and quantity range 1 to 6. That single file-level range cannot exclude September 2. Its Parquet footer has finer row-group ranges, which can exclude the September 1 and 3 groups. This is why Delta file skipping and Parquet row-group pruning are separate opportunities.

A reader is consistent because it uses one committed snapshot, rather than repeatedly discovering a changing collection of data files while executing. It may still fail if someone physically deletes needed files outside the protocol or retention is made unsafe for long readers. Consistency is not permission to remove storage manually.

For our pipeline, Delta lets the nightly output appear as a completed version. It does not verify that quantity 35 is correct, that there is only one row per item-store-day, or that the backfill includes sufficient history. Those are engineering responsibilities. I would explain it as: "Delta gives me a reliable way to publish a new table state. I still build checks for the business grain, reconciliation and safe reruns."

8 Walk the transaction log through six events

The following records are abbreviated teaching extracts. Real commits include required protocol, metadata, paths, sizes, timestamps and other fields. The stats value in an add action is normally serialized JSON text; it is expanded here for readability. This is a view of the idea, not a runnable log-writing recipe. [8]

```

version 0
{"protocol":{"minReaderVersion":1,"minWriterVersion":2}}
{"metaData":{"schemaString":"...", "partitionColumns":[]}}
{"add":{"path":"F0.parquet", "dataChange":true,
  "stats":{"numRecords":4, "minValues":{"qty":1},
    "maxValues":{"qty":4}}}}
{"add":{"path":"F1.parquet", "dataChange":true}}

version 1
{"add":{"path":"F2.parquet", "dataChange":true}}
  
```

```
version 2
{"remove":{"path":"F0.parquet","dataChange":true}}
{"add":{"path":"F0b.parquet","dataChange":true}}
```

Version 0 publishes eight rows in F0 and F1. Version 1 appends the four September 3 rows in F2. Version 2 corrects R02 from quantity 3 to quantity 4 by replacing F0 with F0b in this copy-on-write example. The total changes from 35 to 36. Version 1 still describes the old total if its files and metadata remain available.

Before commit, the writer produces new files and collects task results. Only a successful transaction makes their add/remove actions visible as the next table state. If writing F0b fails before commit, readers continue to see the previous version. An unreferenced physical file may need eventual cleanup, but it is not part of the table simply because it exists.

If two writers read version 1 and both modify the same relevant data, the first may commit version 2. The second validates against intervening changes and can fail with a conflict. It must retry from a fresh snapshot with appropriate application logic. Conflict granularity depends on the operation, isolation and enabled row-level features; two disjoint changes are not universally guaranteed to conflict or succeed. [9, 10]

Later OPTIMIZE may remove F0b, F1 and F2 logically and add F3 containing the same twelve corrected rows. For a pure data-preserving compaction, dataChange is false. The table result stays at total 36. A later eligible VACUUM removes obsolete physical files. It does not recalculate quantities or improve the ordering of active rows. [11]

The documented Delta defaults commonly distinguish seven days of deleted-file retention and thirty days of log retention. These are separate controls, and current table settings may differ. Design retention for the longest reader, delayed stream, recovery and audit needs. Do not turn off retention checks merely to reclaim space, and do not promise thirty days of time travel if needed data files are removed earlier. [11]

Observe DESCRIBE HISTORY and operation metrics. Do not sum every historical file size to calculate the active table size. Time travel needs both a usable historical snapshot and its referenced data. A retained log entry cannot resurrect a physically deleted Parquet file.

9 One correction inside a large file

Suppose R02 is inside a 512 MB Parquet file. Its quantity should be 4, not 3. Parquet is normally immutable in place: the writer does not seek to one byte and alter the row as if it were a database page.

Without a deletion vector, a modification commonly rewrites the affected data file, creating a replacement and removing the original from the active snapshot. A DELETE can instead omit the row from that replacement. If a whole file qualifies for deletion, metadata can sometimes remove it without rewriting its surviving rows because there are none.

With a supported deletion-vector path, metadata can mark the old row position as deleted. For an UPDATE, the corrected row is written elsewhere. A new reader combines the base file with the deletion mask and the replacement data. [12]

Physical object	Before	After the supported update
Base file F0	R01 qty 2, R02 qty 3, R03 qty 4, R04 qty 1	Original bytes may remain
Deletion vector	None	Position for old R02 is masked
Replacement file U0	None	R02 qty 4
Current reader	A S1 Sep 1 total 5	A S1 Sep 1 total 6

The deletion vector is a row-position mask associated with a file, not a business-key index. The actual metadata representation is more detailed than this table. Old snapshots can retain the earlier file state; new readers must support the enabled table features.

MERGE combines matching logic with actions: matched rows may be updated or deleted, and unmatched rows inserted. It does not make a duplicate or ambiguous source correct. Deduplicate using a defined sequence or event timestamp and validate the merge key before writing.

Deletion vectors can defer rewriting work for sparse corrections. The trade-off is read-time handling and future physical cleanup. OPTIMIZE or other rewrites can materialize some changes; when guaranteed purging of soft-

deleted content is required, evaluate the documented REORG TABLE ... APPLY (PURGE) workflow and then retention-safe VACUUM. Do not assume each OPTIMIZE run rewrites every file containing a vector. [12]

For our nightly pipeline, compare write latency, bytes rewritten, read performance and supported downstream clients before enabling a feature. My explanation would be: "For a small correction, rewriting a large file can be expensive. Deletion vectors can avoid some immediate rewrite work, but I check reader compatibility and plan the maintenance as well."

10 From lazy code to jobs stages and tasks

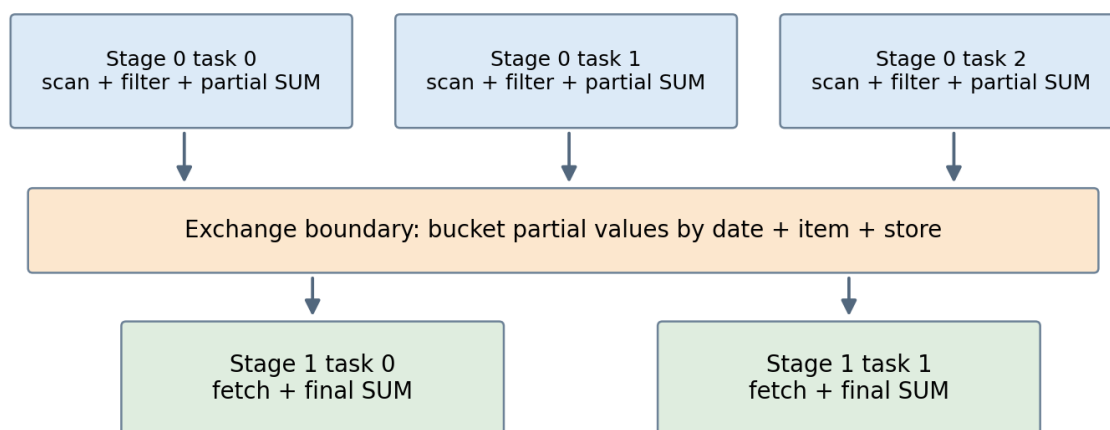
Lazy evaluation means Spark can build a description of transformations without immediately computing their result. Some metadata work can still happen while constructing or inspecting a DataFrame. The main distributed processing is triggered by an action such as a write, count or collection. Avoid collecting 1 TB to the driver.

```
from pyspark.sql import functions as F
sales = spark.table('retail.sales')
selected = sales.where("sale_date >= DATE '2026-09-01'")
selected = selected.select('sale_date', 'item', 'store', 'qty')
daily = selected.groupBy('sale_date', 'item', 'store').agg(
    F.sum('qty').alias('daily_qty'))
daily.write.format('delta').mode('append').saveAsTable('retail.daily')
```

The write triggers execution. A SQL execution may involve multiple Spark jobs, for example because of broadcast preparation or adaptive execution. Do not assume one notebook cell is one Spark job. A stage is a set of tasks that can execute without crossing the next required shuffle dependency. Each task processes one partition of that stage; retries create additional task attempts. [7]

In a conceptual two-stage aggregation, stage 0 reads input partitions, filters and produces local partial sums. Its shuffle output is partitioned by the grouping key. Stage 1 fetches the relevant pieces and computes final sums. A write can add another redistribution depending on the output requirements.

Conceptual aggregation execution



A conceptual stage boundary at the exchange

Suppose three executors each have two usable task slots and each task needs one core. Six tasks can run concurrently in this simplified example. If there are twelve input partitions, there are at least two waves of task scheduling. One slow task may delay the next stage even when most slots have become idle. Real resource profiles, executor availability and scheduler decisions change this picture.

A narrow transformation, such as a simple filter or derived column, can process each partition independently. R01 does not need R02 to decide whether its date matches. A wide transformation needs data across upstream partitions. Computing A-S1's complete daily sum requires finding all rows with that key, wherever they were read.

Do not insert repartition after every transformation. Chaining a scan, filter and projection usually does not require moving each intermediate result across workers. Inspect exchange nodes in the physical plan and shuffle metrics in the UI to identify actual redistribution.

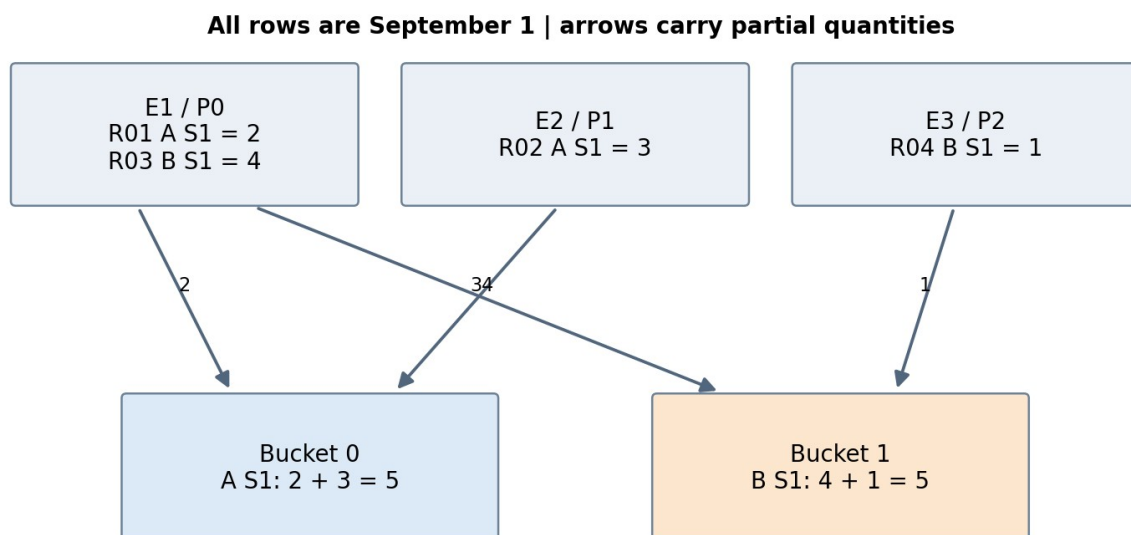
“I think of the driver as coordinating the work and executors as processing the partitions. A shuffle creates a boundary where rows have to be regrouped across workers. That is where I look closely at data size, skew and memory.”

11 Watch the rows move during aggregation

To make movement visible, distribute our September 1 rows across three input partitions. P0 contains R01 and R03, P1 contains R02, and P2 contains R04. A partition may be scheduled on any available executor; the following placement is one execution example.

Input task	Rows before shuffle	Local partial sums
Executor E1 task P0	R01 A S1 qty 2; R03 B S1 qty 4	A S1 = 2; B S1 = 4
Executor E2 task P1	R02 A S1 qty 3	A S1 = 3
Executor E3 task P2	R04 B S1 qty 1	B S1 = 1

All these rows have date September 1. We include the date in the real grouping key. For the drawing only, imagine the partitioning function assigns A-S1-September1 to reducer bucket 0 and B-S1-September1 to bucket 1. Actual hash bucket numbers are implementation-dependent.



Same date + item + store meets in the same reducer bucket

Actual sample quantities moving to final aggregation

The upstream tasks write bucketed shuffle blocks. Downstream tasks obtain their blocks, sometimes locally and sometimes across the network. Bucket 0 combines 2 and 3 to produce 5. Bucket 1 combines 4 and 1 to produce 5. The aggregation result is two rows with total quantity 10, exactly the original four-row total.

Partial aggregation can reduce shuffle when many rows with the same key occur within each input partition. If E1 held a thousand A-S1 rows, it could often emit one partial SUM for that key rather than a thousand original rows. If every row has a different grouping key, local aggregation may reduce little. The correct optimization depends on the operator and actual distribution.

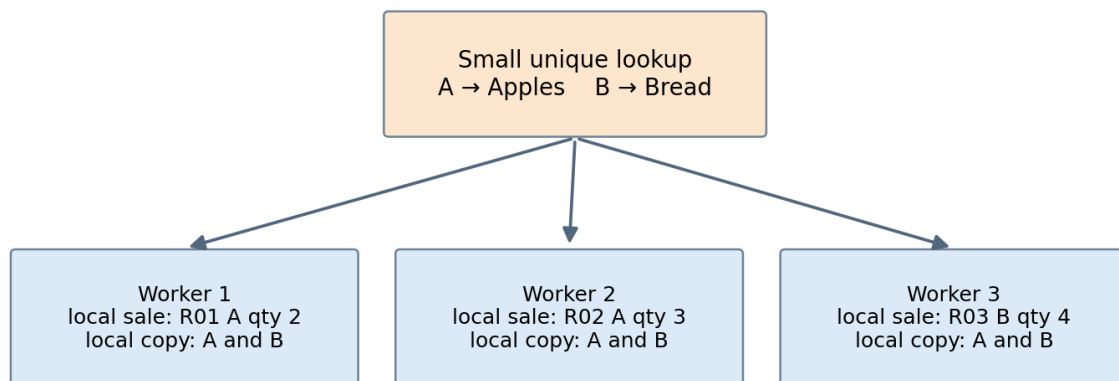
Shuffle write is intermediate data produced upstream. Shuffle read is the data consumed downstream. Remote read is the part obtained from other executors. Spill occurs when an operation writes working data to local disk because it cannot keep the required state in its memory budget. Shuffle storage and spill are related costs, but are not the same metric.

At 1 TB, the shuffle usually need not equal 1 TB. Early filters, column selection and local partial aggregation can reduce it. Conversely, a join that multiplies rows or carries wide unused columns can make intermediate data unexpectedly large. Compare intermediate rows and bytes, not only source table size.

Use a shuffle when the calculation requires it. Avoiding it by grouping each partition independently gives incomplete totals. My explanation: “The shuffle brings the same business key together. I cannot remove that requirement, but I can reduce the rows and columns entering it and check that the work is balanced.”

12 Broadcast a small lookup and shuffle large joins

Our item lookup contains A = Apples and B = Bread. It has one row per item. A broadcast join copies a small build-side relation to workers so tasks can match their sales rows locally. It does not broadcast the 1 TB sales table.



The small lookup travels to workers

R01 matches A once, keeping quantity 2. R02 matches A once, keeping quantity 3. All twelve sales rows remain twelve joined rows with total quantity 35 for this complete unique lookup. The extra description column makes rows wider, but the join does not duplicate them.

Broadcast suitability depends on the filtered relation's in-memory representation, executor resources, supported join type and concurrent work. A small compressed file can decode into a larger hash structure. A forced hint can override size-based caution and cause memory pressure or a timeout. Verify the executed join rather than assuming the hint is always accepted. [2]

For two genuinely large tables, distributing matching keys through shuffle is often appropriate. Before movement, A-S1 records may be spread across both inputs. After hash partitioning by the join keys, matching groups meet in the same join partitions. Sort-merge implementations then order and match keys; other supported engines may use hash-based join operators. The distribution requirement remains even when the local matching algorithm changes.

Now introduce a faulty lookup with two rows for A: Apples and Apples-old, with no effective-date condition. Our twelve-row source contains six A rows and six B rows. The join produces eighteen rows. A's quantity total 21 is counted twice; B's total 14 once. The result sums to 56, not 35.

```
SELECT item, COUNT(*) AS rows_per_key
FROM retail.item_lookup
GROUP BY item
HAVING COUNT(*) > 1;
```

Do not repair this by blindly dropping duplicates. If the table is a history dimension, join on the correct effective-date interval. If multiple matches are legitimate, define the intended output grain and allocation. An arbitrary record selection can hide a business error.

Observe input and output cardinality, unmatched keys, distinct key counts and quantity reconciliation.

Broadcasting the faulty lookup may make the wrong result arrive faster. “Before tuning a join, I check the relationship between the tables. If a lookup unexpectedly multiplies the fact rows, I fix that first. Then I choose the join strategy based on the actual filtered size.”

13 Skew and adaptive execution

Skew means work is unevenly distributed. Suppose most item-store combinations have hundreds of rows, but a default key UNKNOWN-S1 has millions. Hash partitioning sends the same key to the same bucket. Increasing the number of buckets does not automatically split that one key.

In an illustrative stage, 99 tasks finish in two minutes and one takes thirty. Its input is 12 GB while the others read around 200 MB each. First establish whether the hot key is legitimate. It may represent a real high-volume item, an upstream missing-key default, or an erroneous join.

Ordinary equality joins do not match null to null. An inner join can filter null keys, as your screenshots show. A null-safe equality join or a groupBy can gather null keys and create concentration. Do not delete null rows merely to improve performance when they are required for reconciliation.

For a valid SUM, a two-stage salted aggregation can divide a hot key into subkeys, sum each subkey, then sum the partial totals back to the original key. Salt is a temporary distribution label, not a new business key. For AVG, carry SUM and COUNT; averaging partial averages without weights is wrong. Exact distinct calculations and ordered windows need different reasoning.

Illustrative intervention	Before	After
Split valid hot SUM into 8 subkeys	One 30 minute reducer	Eight roughly 4 minute partial reducers
Recombine partials	No second salt-removal step	Small final SUM restores original grain
Correctness check	Total 35 in miniature	Total remains 35

AQE uses runtime statistics to revise eligible physical decisions. It may combine small shuffle partitions, switch an eligible join strategy or split skewed join work. Compare the initial and final executed plans. A static EXPLAIN does not show the final adaptive decisions. AQE is not a general repair for duplicate joins or a guarantee that every hot aggregation key is split. [13]

Avoid salting everything: it adds complexity, extra movement and risk, and joins may require carefully replicating the matching side. Try supported adaptive behavior first when applicable, then isolate the remaining skew. Record task-duration percentiles, largest input partition, spill and post-change reconciliation.

“My first question is whether the hot key is valid. If it is a data-quality problem, I fix the source logic. If it is a real distribution issue, I check AQE and then consider a targeted change that preserves the final business grain.”

14 Repartition coalesce and output parallelism

Repartition creates a new distribution and normally involves shuffle. Coalesce commonly reduces the partition count without a full redistribution. AQE shuffle coalescing is a separate runtime decision over shuffle partitions, not the same API call as DataFrame.coalesce.

Operation	Before in MB	Illustrative after	Main consequence
repartition(4)	80, 10, 5, 5	About 25, 25, 25, 25	Pay shuffle to rebalance rows
coalesce(2)	80, 10, 5, 5	Possibly 90, 10	Fewer tasks, imbalance can remain
repartition(4, item)	A and B mixed	All A together; all B together; empty buckets possible	Same key is colocated, not evenly sized
AQE coalescing	100 tiny shuffle buckets	Fewer combined readers	Reduce task overhead after exchange

The exact grouping in coalesce and exact resulting bytes depend on scheduling, row widths and implementation. Four hash buckets with only two distinct items cannot ensure four busy tasks. Likewise, partitioning a seven-day batch by date alone limits the number of distinct key groups even if a much larger count is requested.

Repartitioning is useful when a severely uneven distribution would harm substantial downstream work, or when you need an explicit distribution for a measured reason. It is wasted when the next operation immediately reshuffles on a different key, or the current distribution is already sufficient.

coalesce(1) makes the final dataset one partition. Writing 1 TB through one task sacrifices parallelism, concentrates risk and can create an unwieldy file. It is appropriate only for genuinely small export requirements that have been measured and bounded. A single logical table does not need to be a single physical file.

Do not equate final partitions with final files universally. A task can write to multiple storage partitions, split files due to limits or participate in optimized writer behavior. Empty partitions can produce no data files. Observe actual output file count, size distribution, writer tasks and elapsed write time.

The practical question is: “What work do I want to distribute, by which key, and for how long will that distribution remain useful?” If I cannot answer that, adding repartition is probably premature. In our small example I can print partition IDs for learning; at production scale I use metrics rather than collecting all rows.

15 Rolling features and the backfill boundary

Our daily table has A-S1 quantities 5 on September 1, 5 on September 2 and 6 on September 3. A three-day inclusive rolling sum on September 3 is 16. If R02 is corrected from 3 to 4, September 1 becomes 6, and the September 3 rolling sum becomes 17.

A feature for predicting today's demand often excludes today's actual sales. The feature definition must say whether it uses days D-7 through D-1 or includes D. Otherwise, a technically correct window can leak future information into training.

```
SELECT sale_date, item, store, daily_qty,
       SUM(daily_qty) OVER (
         PARTITION BY item, store
         ORDER BY day_number
         RANGE BETWEEN 7 PRECEDING AND 1 PRECEDING
       ) AS prior_7_day_qty
FROM daily_with_day_number;
```

Here day_number is an integer business-day index derived consistently from sale_date. RANGE means calendar-day distance on this index. ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING means seven previous rows; it equals seven previous days only if there is exactly one correctly ordered row per day. A missing row is not automatically zero sales. Define the expected item-store-day calendar and distinguish not ranged, store closed, missing feed and genuine zero demand.

Before the window, an item's history may be scattered across tasks. The engine typically redistributes by item and store and sorts by date. Afterwards, each output row carries the window value. A very long or hot series can concentrate work; repeatedly calculating the same window in multiple branches can add expense.

Suppose the publication range is September 10 through 16 and the feature needs the preceding seven days. The read range must start September 3 even though only September 10 through 16 will be written. Apply the output-date restriction after calculating the feature. Filtering to September 10 first would remove needed history.

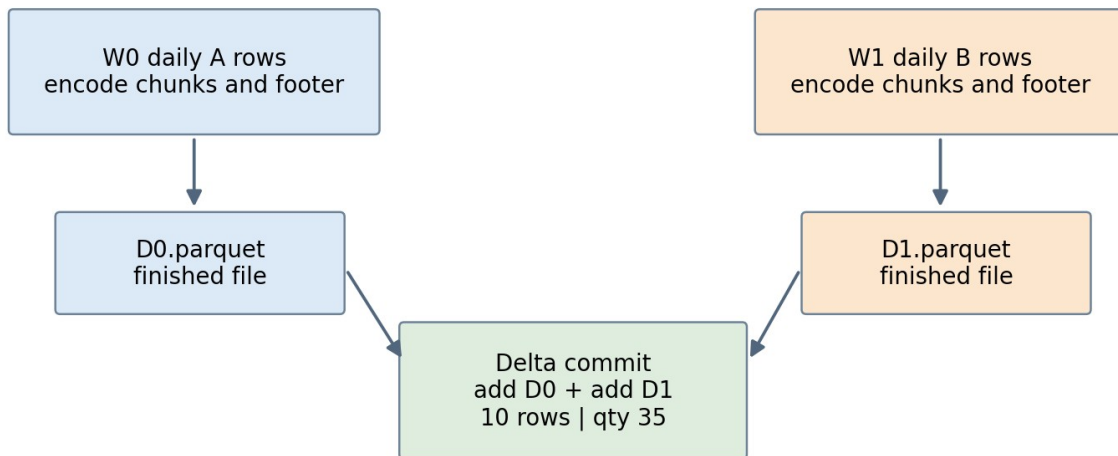
A correction on date C can affect outputs C+1 through C+7 for a prior-seven-day feature. Other features may have a 28-day, 52-week or more complex dependency. Track each feature's dependency horizon and take the required union when planning a repair.

A configuration-controlled rebuild is useful only if the framework separates read_start, output_start and output_end. Store the source snapshot or availability cutoff when reproducibility matters. Event date alone does not prove a historical training row uses only information known at the original prediction time.

“I can drive a rebuild from configuration, but I still calculate the dependency range. For rolling features, I read the extra history and publish only the intended dates. I also check point-in-time availability so the training data does not include information that arrived later.”

16 The writer from batches to committed files

A write is a distributed computation followed by table publication. First the plan establishes the desired rows and distribution. Writer tasks receive their portion, encode column values into pages and row groups, compress the data and finish Parquet files including their footers. Successful task outputs contribute metadata for the Delta transaction.



Parallel files become visible through one commit

In our initial daily output, the twelve transaction rows become ten rows. Imagine task W0 writes A-S1 and A-S2 groups to D0, while W1 writes B-S1 and B-S2 groups to D1. The actual split is illustrative. Once the transaction commits, the Delta snapshot includes D0 and D1 together. A reader does not discover D0 halfway through and call that the new table.

There are two kinds of statistics to keep separate. The Parquet writer records statistics for column chunks in row groups when supported and enabled. The Delta write path collects file-level table statistics for relevant columns. Separately, the optimizer may use table and column statistics to estimate join costs. These are different scopes, even when all involve counts or min/max values.

Many tiny output partitions can produce many small files. Too few large partitions can make a handful of tasks dominate the write. An optimized write can redistribute output to produce more suitable files, adding computation during the write to reduce later read costs. Auto compaction combines eligible small files after writes. Their exact enablement and behavior depend on compute, runtime and table type. [14]

A target file size is a target for compressed output, not a promise of an exact row count or a fixed amount of executor memory. A million short integer rows differ from a million rows containing long text. Plan file sizes from observed output, not an assumed universal rows-per-file ratio.

If a task fails, Spark may retry it. The table transaction must publish the intended successful outputs rather than every abandoned task file. This does not give arbitrary side effects in a UDF exactly-once behavior. Sending external messages or modifying another system inside a retried task requires separate idempotency design.

Observe write task distribution, file sizes, output rows, commit duration and Delta history. “I look at the write as two parts: producing suitable files in parallel and publishing one consistent table state. I measure both, because a slow commit and a slow writer need different fixes.”

17 Choose append merge or date range replacement

The best write strategy depends on what the batch represents, not just on which SQL command is shortest.

Situation	Candidate	Correctness condition
New immutable records only	Append	Replay cannot append the same business records twice
Sparse corrections and new keys	MERGE	One unambiguous source row per target key
Complete recomputation of a known date interval	Predicate replacement	The batch contains every intended row for that interval
Entire table rebuilt	Controlled full replacement	All expected history present and downstream impact planned

Appending R09 to R12 to a table containing R01 to R08 is appropriate once. Rerunning that append produces duplicates unless the write or application tracks the batch safely. Delta's atomic commit does not automatically make an arbitrary repeated append idempotent.

For daily output, a MERGE on item, store and sale_date can update A-S1-September1 from 5 to 6. The merge must also update affected rolling-feature dates. Filtering the source alone may not adequately restrict the target search; use valid target predicates without excluding legitimate matches. Check the executed plan and touched files.

```
(recomputed_output.write.format('delta')
  .mode('overwrite')
  .option('replaceWhere',
    "sale_date >= '2026-09-10' AND sale_date < '2026-09-17'")
  .saveAsTable('retail.forecast_features'))
```

This is a bounded replacement pattern. Validate completeness, dates and unique keys before the write. Use the runtime's documented predicate validation behavior; do not disable safeguards simply to get a batch through. A missing store in the recomputed batch can otherwise disappear from the replaced interval. [15]

For historical rebuilds, keep publication boundaries explicit and consider staging plus validation before exposing the result. Record run ID, input versions, config, row counts, checksums or reconciled measures, and output version. Advance last_successful_date only after publication succeeds. If a failure occurs after publication but before the control update, the retry must recognize or safely reproduce the committed output.

For forecasting, validate uniqueness at item-store-day, quantity reconciliation where meaningful, null rules, expected date coverage, and feature values for known examples. The pre-correction daily total is 35; after the one-unit correction it is 36. Compaction should change neither total.

"I use append for genuinely new immutable data, MERGE for key-based changes, and date-range replacement when I can reliably rebuild the complete interval. For me, the main decision is the rerun and correction behavior, not just write speed."

18 Diagnose reading and transformation costs

Start with a fixed input snapshot and a correctness baseline. Separate startup, queueing, planning and execution. A slow user experience can come from waiting for compute even when the scan itself is efficient.

Symptom and evidence	Likely causes to test	Change and trade-off
Need 7 days but scan nearly 1 TB	Missing date predicate, overlapping file statistics, unsuitable layout	Push a valid date restriction down; improve localization. Extra maintenance may be needed
One huge Parquet file	Too few row groups or poor scan parallelism	Inspect splits and groups; rewrite once into suitable pieces. A huge file is not automatically one task
100000 tiny files	Frequent small appends or excessive write partitions	Compact eligible files and fix write pattern. Compaction itself reads and writes data
Narrow SELECT still reads many bytes	Wide selected columns, filter columns, weak pruning	Inspect scan schema and actual bytes; avoid unused columns before shuffle
Selective filter reads every row group	Mixed values or unsupported predicate/statistics	Improve layout or use a supported direct predicate; compare groups and bytes
Driver planning is slow	File/partition metadata volume, huge query plan	Reduce small files or simplify repeated branches; do not collect source data to the driver

A Parquet file can be splittable while a compressed CSV file may not be usefully splittable in the same way. Do not apply a generic "one file equals one task" rule to both. Inspect the source format and the physical scan.

If a transformation is slow, first inspect its input size and business logic. Carrying forty columns into a join when only six are needed inflates movement. Aggregating transaction lines to daily totals before joining a unique daily-compatible lookup can reduce work, but it is invalid if the lookup changes the classification of individual transactions.

Transformation evidence	Investigate	Avoid
Huge join shuffle	Actual filtered sizes, join keys, uniqueness, unnecessary columns	Broadcasting a relation just because it is called a dimension
Aggregate spill	Group cardinality, partition sizes, concurrent memory use	Increasing memory without examining a row explosion
One long task	Hot keys, skew, host failures, uneven input	Assuming every straggler is data skew
Python evaluation dominates	UDF implementation and data transfer	Expecting Photon to execute arbitrary Python code
Repeated identical scans	Reused branch and cache feasibility	Caching 1 TB with no reuse or insufficient capacity

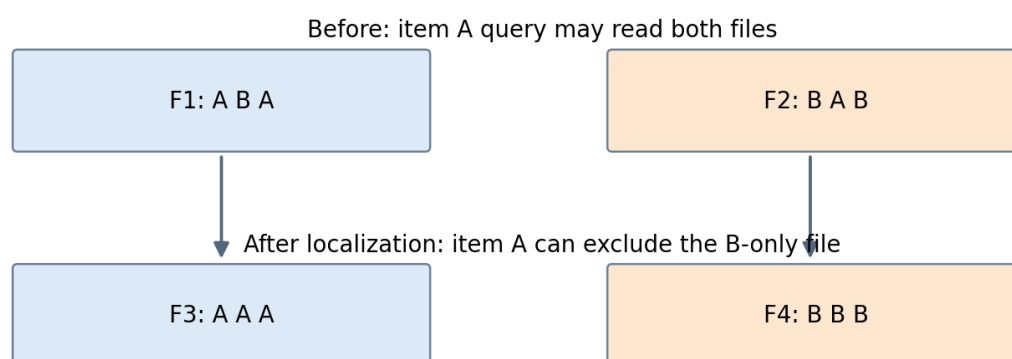
For a repeated small filtered dataset, caching can avoid recomputation. The first action still populates the cache, and memory competes with execution. A Spark persisted DataFrame, a storage-layer disk cache and a SQL result cache are different mechanisms. Benchmark cold and warm runs separately and avoid accidentally comparing a real run to a cached answer.

“I follow the expensive operator back to its input. A large shuffle may be caused by a missing filter or duplicate join much earlier. I fix that cause before tuning the operator in isolation.”

19 Layout and maintenance with before and after files

Compaction changes how rows are packaged. Clustering changes which values tend to live together. Retention cleanup removes eligible obsolete files. Optimizer statistics help planning. These actions solve different problems.

Same six illustrative rows, different file locality



Localization makes file statistics more selective

Operation	Before and after in our example	Use and limitation
OPTIMIZE compaction	Ten 10 MB active files become roughly one 100 MB file	Reduce opens; may reduce parallelism or skipping if taken too far
Date partitioning	Mixed-date files become separate date directories	Useful for date-scoped work; many tiny dates or high-cardinality keys can fragment storage
ZORDER by item	Item ranges overlap widely, then become more localized	Improve skipping for selective item predicates; rewrite cost and diminishing benefit with many keys
Liquid clustering	Flexible clustering keys guide incremental layout	Alternative to partitioning and ZORDER; support and protocol affect clients
Deletion vectors	Rewrite immediate file, or defer via row mask	Sparse changes may cost less now; reads and maintenance must support masks

Operation	Before and after in our example	Use and limitation
ANALYZE	Unknown/stale estimates become better planning statistics	Does not physically sort or compact the rows
VACUUM	Obsolete FO retained, then physically removed when eligible	Reclaims space; reduces recoverable history, not active-file fragmentation

Traditional date partitioning and ZORDER on other useful columns can be combined. Liquid clustering is an alternative layout strategy; do not add ZORDER or traditional partitioning to a liquid-clustered table as if they were independent switches. Choosing item and date as keys does not guarantee strict global sort order. Evaluate representative predicates and actual file skipping. [16]

For our 1 TB table, date partitioning may work well if each day is substantial and jobs mostly rebuild date intervals. If queries vary between date, item and store predicates, liquid clustering may offer useful flexibility. Existing jobs may depend on partition overwrite semantics or physical paths, so migration includes validation and operational changes, not only an ALTER TABLE.

Current documentation describes direct partition-to-liquid conversion in supported newer runtimes, including Runtime 18.1 and above. Do not assume that syntax exists in the user's deployed runtime. Alternatives include building and validating a new clustered table, backfilling it, then planning a controlled consumer cutover. Evaluate protocol compatibility, storage during migration, cost, rollback and whether existing history must remain accessible. [16]

Predictive optimization can manage eligible maintenance for Unity Catalog managed tables, including OPTIMIZE, VACUUM and ANALYZE. Check eligibility, effective enablement, retention and billed work rather than scheduling redundant manual jobs. It is not a universal automatic service for every external Parquet folder. [17]

Use history to identify files added/removed and maintenance frequency. Measure a representative read/write mix before and after. A layout that makes one dashboard faster can make frequent corrections more expensive. Do not run maintenance on a fixed schedule without checking whether it improves enough work to justify its cost.

20 Compute and settings without guessing

Start with the effective configuration, because Apache Spark defaults are not a universal description of Databricks SQL warehouses or serverless compute. Some settings are managed or unavailable on particular compute types. Record runtime, engine, access mode, worker shape and table features with each benchmark.

Control	Documented reference value or scope	Review when and how to verify
spark.sql.shuffle.partitions	Spark 3.5.5 reference 200; Databricks documentation also lists 200 and supports auto where applicable	Too many tiny reducers or too few large ones; inspect final tasks, shuffle sizes and spill
spark.sql.autoBroadcastJoinThreshold	Spark 3.5.5 reference 10 MiB	Small unique relation shuffles; inspect estimates and executed join, not just threshold
spark.sql.adaptive.enabled	Apache Spark AQE enabled by default since 3.2	Adaptive behavior unexpectedly absent; inspect final plan
spark.databricks.optimizer.adaptive.enabled	Databricks AQE documentation true	Check applicable runtime and final adaptive plan
spark.databricks.adaptive.autoBroadcastJoinThreshold	Databricks adaptive reference 30 MB	Eligible runtime join remains shuffled; check actual build-side size and join type
spark.sql.adaptive.advisoryPartitionSizeInBytes	Reference 64 MB	Small tasks or oversized post-shuffle work; advisory size is not a file size guarantee
spark.sql.adaptive.skewJoin.enabled	Reference true	Eligible skewed joins; verify supported join side and actual skew handling
spark.sql.files.maxPartitionBytes	Spark reference 128 MiB for file scan packing	Scan partitions too large/small; it does not rewrite Parquet files

Control	Documented reference value or scope	Review when and how to verify
<code>spark.sql.files.openCostInBytes</code>	Spark reference 4 MiB estimate	Packing many small files; this is a planning cost estimate, not measured request bytes

Reference values above are version-scoped documentation values, not instructions to set them all. Inspect with `spark.conf.get` where the compute supports it; use compute configuration and executed metrics for managed SQL/serverless behavior. [2, 13]

More cores allow more simultaneous tasks when there is sufficient independent work. More executor memory can help state-heavy operations, but increasing concurrent tasks on the same executor can consume that memory again. A single hot reducer does not become parallel simply because the cluster doubles.

Autoscaling can respond to changing workloads, but scale-up takes time and may not help a short burst. Job compute provides a controlled execution environment for scheduled work; SQL warehouses suit SQL-centric workloads and concurrent SQL serving. Compare startup, queueing, utilization and billing for the actual schedule. Avoid treating “serverless” as “free” or “automatically the cheapest.”

Review broadcast hints only after checking uniqueness, filtered size and memory. Review caching only when reused computation justifies materialization. Review write tuning based on output file distributions and runtime-specific automatic features. Override one control at a time, preserve the baseline, and roll back changes that only move cost elsewhere.

21 The same query with Spark and Photon

Run the same filtered sales-to-item join followed by daily aggregation and a Delta write. The business result must remain the ten daily rows and total 35 before correction. Photon is an execution engine within Databricks, not a replacement table format or a different business query language.

Aspect	Standard Spark SQL execution	Photon enabled for supported work
Logical request	Filter, join, aggregate, write	Same logical request
Planning	Spark SQL planning and optimization	Catalyst planning with supported native execution
Reader and operators	JVM execution with supported vectorized reading and code generation	Native vectorized operators and supported reader/writer paths
Distribution	Tasks across workers; exchanges when required	Still distributed; required exchanges remain
Custom Python function	Runs outside ordinary SQL native operators	Not converted into native Photon code
Output	Delta snapshot over Parquet files	Same table semantics

For R01 and R02, both paths must produce A-S1-September1 quantity 5. Photon can process batches through supported native operators efficiently. Unsupported portions fall back as documented. A Python notebook using built-in DataFrame expressions can benefit because those expressions describe engine operations; that is different from arbitrary Python row processing. [18]

Workload	Decision to test	Evidence that matters
SQL-heavy Range Optimiser joins and aggregates	Strong Photon benchmark candidate	Operator coverage, execution time and full cost
Feature pipeline using built-in DataFrames	Strong candidate for supported operations	Final plan, Photon task share, correctness
Custom Python dominates runtime	Fix or redesign costly Python where practical	Python time and transition cost
Avoidable full scan dominates	Fix pruning, then benchmark engine	Files and bytes scanned before speed claims
Very short job	Measure startup and planning share	End-to-end duration and billed minimums

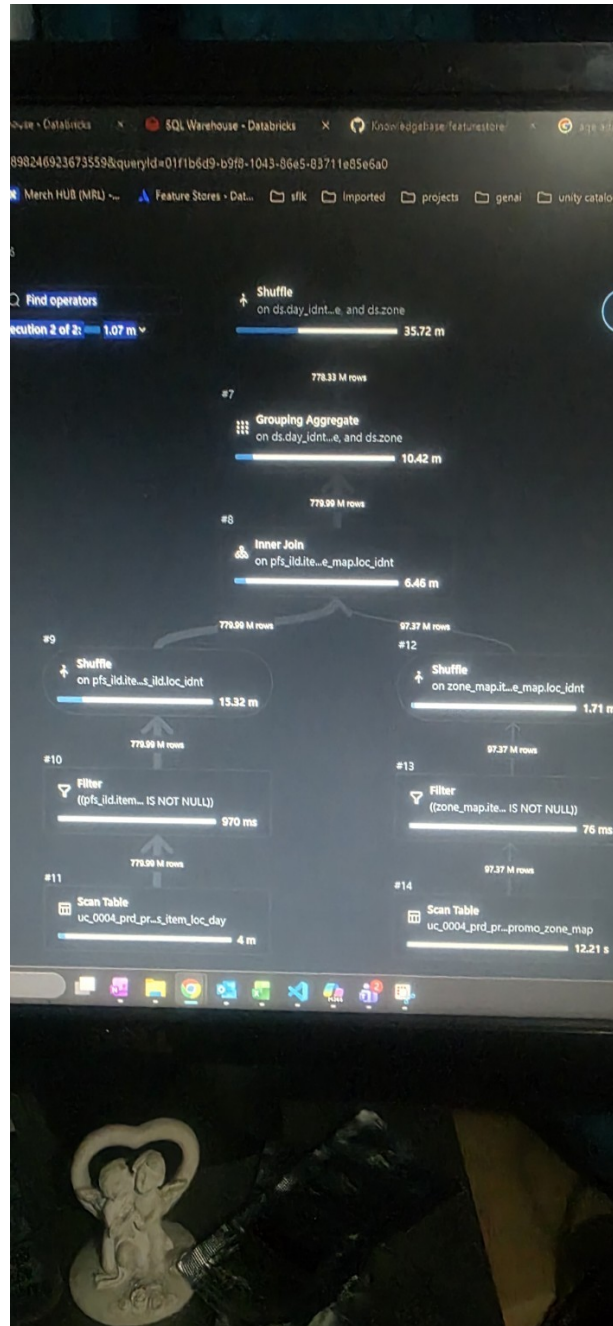
Use repeated comparable runs with the same source snapshot, workload, cache state and concurrency. Record elapsed time, startup, queueing, compute charges, bytes read, shuffle, spill, output rows and feature checks. Different engine paths can change floating-point reduction order; use appropriate numeric tolerances rather than declaring every tiny floating-point difference a business error.

Illustrative cost arithmetic: a 30-minute run at 6 cost-units per hour costs 3 units. A 20-minute run at 10 units per hour costs about 3.33 units. Faster can cost more. Conversely, higher hourly compute can cost less if it finishes sufficiently sooner. Use actual billing rates and include applicable infrastructure and platform charges.

“I would expect Photon to be useful for our SQL-heavy transformation work, but I would demonstrate that with a like-for-like run. I check the total cost and the executed operators, not just whether the Photon option is enabled.”

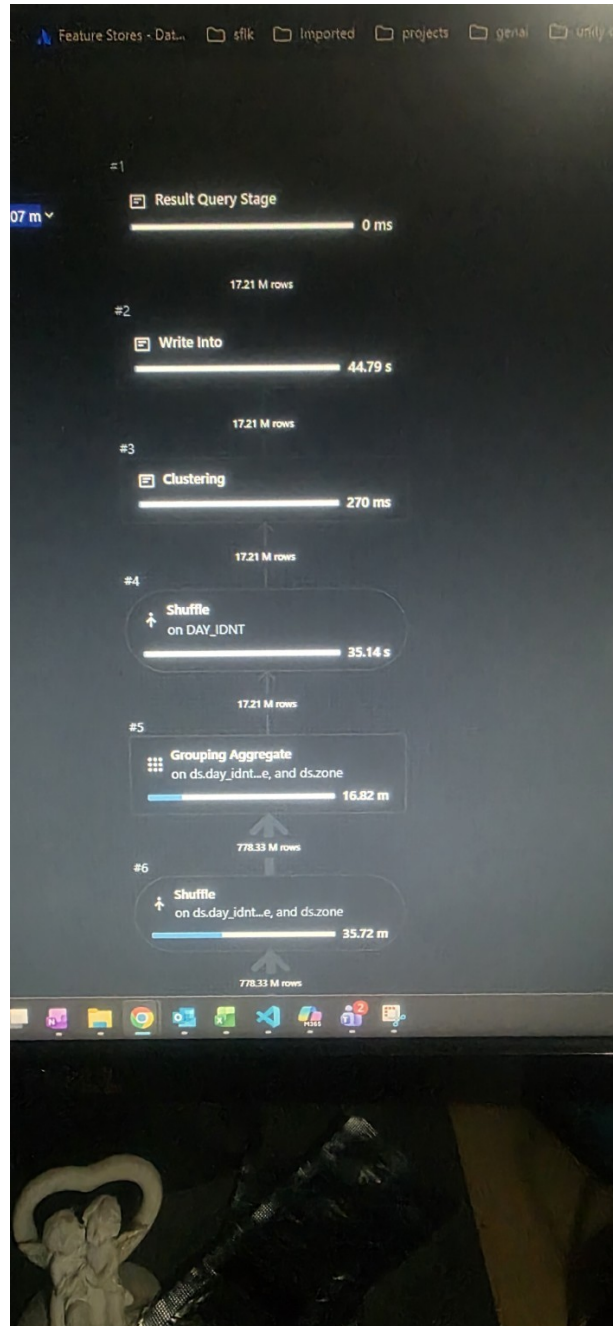
22 Your two query profile photographs

These photographs show executed query-profile operators and row counts. They provide more evidence than a static EXPLAIN, but they do not contain the full SQL, all scan details, task distributions or compute configuration. Some identifiers are truncated in the photographs. The transcription below preserves visible values without guessing complete names.



Supplied photograph 1 Read the plan upwards from the scans

The first image is the lower part of the plan. Read the arrows from the scans upwards. The sales-side scan is labelled with an item_loc_day ending and approximately 779.99 million rows. The mapping-side scan ends in promo_zone_map and shows 97.37 million rows. Both sides have IS NOT NULL filters, then shuffles before the inner join.



Supplied photograph 2 Read the plan upwards from the scans

The second image continues the upper path: grouping, shuffle, final grouping, a DAY_IDNT shuffle, clustering, write and result. The shared shuffle node 6 links the two photographs. The images are a useful case study, but a photograph is not enough to identify a proven bottleneck or prescribe a safe broadcast hint.

23 Walk your profile from scans to the write

Node	Visible operation	Visible figure	What to check next
11	Sales-side table scan	About 779.99 M rows; 4 m	Bytes, files read/pruned, date predicate and scan schema
14	Promotion zone mapping scan	97.37 M rows; 12.21 s	Effective grain, date restrictions and actual byte size
10 and 13	IS NOT NULL filters	970 ms and 76 ms	Whether null removal follows inner-join semantics
9 and 12	Shuffles on join-related keys	15.32 m and 1.71 m	Shuffle bytes, key distribution and join conditions

Node	Visible operation	Visible figure	What to check next
8	Inner join	About 779.99 M output rows; 6.46 m	Matches, missing keys and uniqueness at exact join grain
7	Grouping aggregate	778.33 M output rows; 10.42 m	Partial versus final aggregation and grouping keys
6	Shuffle on grouping keys	778.33 M rows; 35.72 m	Bytes, spills, task spread and local reduction
5	Grouping aggregate	17.21 M output rows; 16.82 m	Final groups and aggregation state
4	Shuffle on DAY_IDNT	17.21 M rows; 35.14 s	Distribution required by write or layout
3	Clustering	270 ms	Operator details; not proof of liquid clustering
2	Write Into	17.21 M rows; 44.79 s	Output files, sizes and target definition
1	Result Query Stage	0 ms shown	Wrapper/result behavior, not evidence that writing took no time

A crucial timing distinction: operator times can represent combined task work across cores. They are not necessarily serial wall-clock durations. The visible roughly 1.07 m near the execution selector is a different UI context; confirm its label in the live profile before interpreting it. Do not add 35.72 m, 16.82 m and the other node times and report that sum as the query runtime. [19]

The two aggregates around node 6 are consistent with partial aggregation followed by redistribution and final aggregation. Confirm the operator details because the photographs alone do not prove the exact implementation. The approximate reduction before that shuffle is only 1.66 M rows, or 0.21% of 779.99 M. The final reduction from 778.33 M to 17.21 M is about 45.2 to one. That makes the grouping keys and local distribution worth investigating.

The join output is close to the visible sales input. That does not prove uniqueness: duplicate matches and unmatched rows could offset each other. Validate the exact item-location-zone-time relationship. Also, 97.37 M rows is not a small lookup just because it is called a mapping table. Measure its projected, filtered byte size before considering broadcast.

A node called Clustering in a write plan does not establish that the Delta table uses liquid clustering. Inspect DESCRIBE DETAIL, SHOW CREATE TABLE and the write command. Likewise, a shuffle by DAY_IDNT does not prove that exactly one file is produced per date.

24 What I would investigate in your query

First capture the SQL text, final executed plan, query duration, compute size, Photon coverage and operator details for this execution. Exporting the profile is better than relying on cropped photographs. Preserve an unchanged baseline and a representative input snapshot.

Read layer: check whether the sales scan should include all 779.99 M rows. If the job needs a bounded date range, verify that the range reaches the scan. For promotion-zone mapping, inspect effective dates and selected columns. A mapping with historical versions may need a temporal predicate as well as item and location keys.

Join layer: validate uniqueness at the precise join grain. Check unmatched sales keys and whether multiple zone assignments are expected. If assignment can change within a day, verify the intended daily allocation. An optimization that aggregates too early can destroy this information.

Aggregation layer: inspect why the partial aggregate reduces so little. High local key cardinality may be normal. If the business logic permits, reduce transaction-level data before a wide mapping join, or avoid carrying unnecessary dimensions in the grouping key. Compare shuffle bytes as well as rows: 778 M narrow partial states differ from 778 M wide rows.

Execution layer: check the maximum and median task duration, spill, shuffle read and executor utilization for node 6 and its stage. A high total task time may be broad parallel work rather than one long tail. More partitions help a generally oversized workload differently from a single hot key.

Write layer: inspect the target layout and why data is redistributed by DAY_IDNT. The write appears much smaller than the upstream aggregate in row count, but its elapsed contribution must be measured. Do not remove a writer exchange just because it looks unnecessary in the screenshot.

Tempting changes I would reject without evidence include broadcasting the full mapping, coalesce(1), increasing the cluster only because one node says 35.72 m, and claiming the clustering operator proves liquid clustering. Each of those confuses a visible label with the physical requirement.

A natural explanation for a review meeting is: “The main area I would investigate is the data going into the aggregation shuffle. There are still around 778 million rows at that point, and the final output is around 17 million. I would check the grouping logic, mapping grain and shuffle metrics first. I would also confirm the timing view, because these operator times are not necessarily the elapsed time of the query.”

25 One complete 1 TB troubleshooting case

This is a separate invented case, not a measured result from your screenshots. All numbers below are illustrative and internally tied to one fixed seven-day workload. The source contains one year of history in 1 TB of active files. The target has 17 million valid item-store-day rows. Each run uses the same input snapshot, produces the same correct values and uses comparable cold-cache conditions.

The initial pipeline scans excessive history, carries unused columns, uses an unnecessarily early wide join and creates many small output files. There is no duplicate-key error in this particular case; fixing a correctness defect would require a new validated baseline rather than claiming equal output.

Cumulative run	Single change introduced	Scan GB	Shuffle GB	Spill GB	Elapsed min
A	Baseline	900	180	80	48
B	Move valid date restriction to the scan	25	180	80	32
C	Project only needed columns before join	12	95	30	26
D	Aggregate at a valid earlier grain	12	24	4	18
E	Enable suitable automatic shuffle sizing	12	24	0	15
F	Improve write sizing	12	26	0	14
G	Test Photon on supported compute	12	26	0	10

Run B changes scan work but not downstream shuffle because the old query eventually applied the same filter after reading. Run C reduces column bytes without changing row counts. Run D is valid only because the mapping is unique and stable at the early aggregate grain. Run F adds a little write redistribution, but reduces output files from 12000 to 240 for an illustrative 60 GB result, around 250 MB per file.

Maintenance is evaluated separately after F: a candidate layout reduces a representative selective read from 12 GB to 5 GB. Its one-off maintenance cost is 20 cost-units, and measured saving per representative run is 0.5 units. It needs about forty such runs to recover that cost, before considering storage and future maintenance. This is not folded into G, so G remains an engine comparison with the same layout.

For A through F, a simplified all-in compute rate of 6 units per hour gives costs 4.8, 3.2, 2.6, 1.8, 1.5 and 1.4 units. At 8 units per hour, G's ten minutes costs about 1.33 units. These are arithmetic examples, not Databricks prices. Actual autoscaling, startup, billing granularity and concurrent usage require real billing data.

Reject a forced broadcast of a 9 GB decoded relation: replicating it on every worker risks memory pressure. Reject reducing to one output task even if it makes one neat file. Validate row counts, keys, sums and rolling-feature samples after every change. Report both performance and correctness evidence.

26 Twenty interview questions with follow ups

1 How do you begin optimizing a pipeline

Testing: structured diagnosis and ownership. Short answer: "I start with the business grain and the expected result. Then I follow the executed plan from reading through to writing and identify where the time and data movement are going."

Deeper answer: For the seven-day job, compare selected dates with scan bytes before considering compute. Then validate lookup uniqueness and intermediate row counts. Challenge: a fast cached run can hide the real workload. Evidence: fixed input version, final plan, timing breakdown and reconciled output.

2 What is the difference between a file and a partition

Testing: storage versus execution understanding. Short answer: "A file is a stored object. A Spark partition is a piece of work at a stage. They are related during a scan, but they are not necessarily one to one."

Deeper answer: F0 can contain multiple row groups; small files may be packed into scan tasks, and a shuffle creates a new distribution unrelated to the original file boundaries. Challenge: assuming one executor per file. Evidence: file count, scan partitions and task count.

3 Why is Parquet useful for analytics

Testing: columnar storage reasoning. Short answer: "It lets us read the columns we need, and similar values often compress well. Statistics can also help the reader avoid irrelevant row groups."

Deeper answer: A September 2 query can skip the other two row groups in our real sample. Challenge: mixing every date into every group prevents that pruning. Evidence: scan schema, footer statistics and bytes actually read.

4 What does the footer contain

Testing: reader mechanics. Short answer: "It contains metadata that helps the reader locate and interpret the columns and row groups. It is not the actual table data."

Deeper answer: The reader locates serialized metadata using the file tail and can find the quantity chunk for the retained group. Challenge: treating footer min/max as an exact index of every value. Evidence: extracted row-group metadata, offsets and actual filter result.

5 Why Delta instead of just Parquet

Testing: transaction and table semantics. Short answer: "Parquet is the file format. Delta adds a committed view of which files make up the table, along with schema and transaction behavior."

Deeper answer: Version 2 swaps F0 for F0b without exposing a half-completed replacement. Challenge: Delta does not guarantee our item-store-day key is unique. Evidence: history, active snapshot and validation results.

6 What happens when a write fails

Testing: failure recovery. Short answer: "If it fails before the transaction commits, the new files do not become part of the table. I still need a safe retry because the control process may fail after a successful commit."

Deeper answer: A failed F0b write leaves version 1 visible. An output commit followed by a failed watermark update is a different case. Challenge: retrying an append blindly. Evidence: run ID, committed version, control state and duplicate checks.

7 What causes a shuffle

Testing: distribution requirements. Short answer: “A shuffle is needed when matching or grouping rows requires them to meet across partitions.”

Deeper answer: R01 and R02 must reach the same grouping destination to sum to 5. Local partial aggregation can reduce transfer. Challenge: removing the shuffle by producing partition-local totals only. Evidence: exchanges, shuffle bytes and final reconciliation.

8 How do you diagnose skew

Testing: evidence-led performance work. Short answer: “I compare the slowest tasks with typical tasks and check whether a few keys hold most of the data. Then I check whether those keys are valid.”

Deeper answer: A thirty-minute task among two-minute peers may indicate a hot key, but failures or a slow host are also possible. Challenge: increasing partition count cannot automatically split one hash key. Evidence: task distribution, partition bytes, key frequencies and retries.

9 When would you broadcast a table

Testing: memory and join judgment. Short answer: “When the filtered lookup is genuinely small enough for the workers and its join relationship is correct.”

Deeper answer: Broadcast A and B descriptions so sales rows match locally. Challenge: a small compressed input can expand into a much larger hash structure. Evidence: projected relation size, supported join strategy, executor memory and final plan.

10 How do you detect a bad join

Testing: correctness before speed. Short answer: “I check the expected match relationship and compare rows and measures before and after the join.”

Deeper answer: Two A lookup rows turn twelve source rows into eighteen and quantity 35 into 56. A history lookup may need effective dates rather than arbitrary deduplication. Challenge: aggregate input/output counts can conceal offsetting losses and duplicates. Evidence: per-key match counts, unmatched records and measure reconciliation.

11 What does AQE do

Testing: adaptive planning scope. Short answer: “It uses runtime information to adjust eligible execution decisions, such as shuffle readers or join strategies.”

Deeper answer: A planned shuffle join may change when the filtered lookup proves small; tiny shuffle partitions may be combined. Challenge: it does not repair a wrong join or guarantee every aggregate is balanced. Evidence: initial versus final plan, actual partition sizes and operator changes.

12 Repartition or coalesce

Testing: movement versus parallelism. Short answer: “Repartition pays for redistribution. Coalesce can reduce partitions more cheaply, but can leave the work uneven.”

Deeper answer: 80,10,5,5 MB may become a poor 90,10 split with coalesce; repartition can distribute rows more evenly. Challenge: keyed repartition stays skewed when one key dominates. Evidence: task input distribution, shuffle cost and downstream savings.

13 How do you choose output file sizes

Testing: workload trade-offs. Short answer: "I look at read patterns and output volume. I want to avoid lots of tiny files without reducing useful parallelism too far."

Deeper answer: Repeated small appends can produce thousands of files; optimized writes or compaction can help. Challenge: one universal 128 MB or 1 GB target ignores selectivity and row width. Evidence: file-size percentiles, scan time, write time and maintenance cost.

14 How do you choose a layout

Testing: architecture judgment. Short answer: "I start with the common filters and the update pattern, then compare layouts using representative workloads."

Deeper answer: Date partitioning may suit complete daily rebuilds; selective item and store access may justify clustering. Challenge: liquid clustering changes compatibility and operational assumptions and is not combined with traditional partitioning or ZORDER. Evidence: predicate patterns, skipped bytes, write amplification and migration checks.

15 What is the difference between OPTIMIZE and VACUUM

Testing: maintenance semantics. Short answer: "OPTIMIZE changes active file organization. VACUUM removes eligible old files that are no longer needed within the retention policy."

Deeper answer: Compact F0b, F1 and F2 into F3 without changing total 36, then later clean obsolete physical objects. Challenge: aggressive cleanup can break historical reads or long-running operations. Evidence: history, active file count, retention and recovery requirements.

16 When are deletion vectors useful

Testing: update mechanics. Short answer: "They can reduce immediate rewriting for small changes by marking old row positions and applying that mask when reading."

Deeper answer: Correct R02 using a mask plus a replacement row instead of immediately rewriting a large base file. Challenge: compatibility and eventual physical cleanup still matter. Evidence: table features, bytes rewritten, downstream reader tests and maintenance behavior.

17 How would you evaluate Photon

Testing: engine knowledge and cost accountability. Short answer: "I would benchmark the same workload and check which operators use Photon, along with elapsed time and total cost."

Deeper answer: Our SQL join and aggregate are good candidates; arbitrary Python work is not automatically accelerated. Challenge: avoidable full scans remain avoidable. Evidence: operator coverage, repeated runs, cache controls, billing and correctness.

18 How do you make a backfill safe

Testing: operational design. Short answer: "I separate the range I need to read from the range I intend to publish, validate the output and make the rerun behavior explicit."

Deeper answer: September 10 output with a prior-seven-day feature requires earlier input. Late corrections affect downstream feature dates. Challenge: advancing the last-successful parameter before commit can skip data. Evidence: configuration, dependency horizon, output version and boundary tests.

19 Would you add more compute to reduce spill

Testing: appropriate-layer tuning. Short answer: "Possibly, but first I check whether the amount of data and the distribution are reasonable. More memory should solve a measured need."

Deeper answer: Project unused columns away and fix accidental join multiplication before scaling. Increase parallelism or memory only if the valid workload still needs it. Challenge: more concurrent tasks can consume the extra memory. Evidence: spill, task sizes, executor utilization and cost per successful run.

20 How do you explain this profile to a stakeholder

Testing: clear communication with uncertainty. Short answer: "The largest area to investigate is the aggregation shuffle. I need the detailed metrics to tell whether the issue is data volume, imbalance or memory."

Deeper answer: The profile shows around 778 M rows before the exchange and 17.21 M after the final aggregate. Challenge: operator task time must not be mistaken for wall-clock duration. Evidence: full query, task distributions, bytes, final plan and measured improvement.

27 A two minute spoken explanation

"One of the main things I do when optimizing a Databricks pipeline is follow the data from the source through to the final output.

I start by checking what the job actually needs to produce. For example, in a forecasting pipeline, the output may be at item, store and day level. Before changing anything, I check that the joins preserve that grain and that the totals are correct.

Then I look at the read. If the source has a terabyte of history but the job only needs seven days, I check whether we are actually reading those seven days. I look at the filters, the columns selected, the file layout and how much data gets skipped.

Next I check the joins and aggregations. I look at the number of rows going into each step, the amount of shuffle, whether tasks are balanced, and whether there is spill. If a lookup unexpectedly multiplies the rows, I fix that before tuning. If a lookup is genuinely small, I consider broadcast. For large joins, I focus on reducing and distributing the valid work.

On the write side, I check the load pattern and file sizes. I choose between append, merge and a controlled date-range replacement based on how corrections and reruns should work. For rolling features, I also make sure the read includes enough earlier history.

After that, I look at compute and Photon. I compare the same workload and check both the runtime and the total cost. I change one thing at a time and validate the output after each change, so I can explain exactly what improved and why."

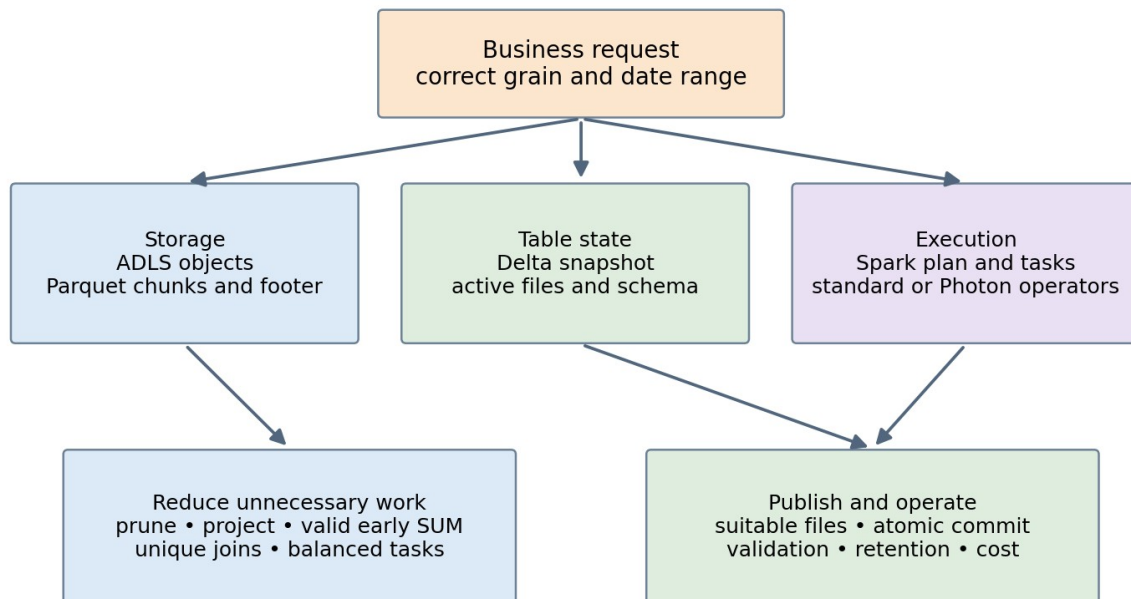
Short version: "I check correctness first, then follow the read, joins, aggregation and write. I reduce unnecessary data and movement before tuning compute. I prove the improvement with the executed plan, runtime, cost and output validation."

28 One page diagnostic checklist

Use this page beside the executed plan. Check each item only when there is evidence.

Layer	Questions to answer
Baseline	What source version, date range, runtime, engine and compute were used? Is time spent in queueing, startup, planning or execution?
Correctness	What is one source row and one output row? Are keys unique where required? Do totals and expected coverage reconcile?
Read	Which columns are scanned? How many files and bytes are read or skipped? Does the intended date predicate reach the source?
Parquet	Are row groups suitably sized and values localized? Are useful statistics present? Does the actual reader support the relevant pruning?
Join	Is each join one-to-one, many-to-one or intentionally many-to-many? Are temporal conditions complete? What is the filtered build-side size?
Shuffle	How many bytes move? Is the largest partition far larger than typical? Are unused columns or redundant rows being transferred?
Aggregate	Can a correct earlier aggregation reduce data? Is grouping cardinality expected? Is spill broad or concentrated?
Windows	Is date order correct? Are missing days defined? Does the read include the dependency horizon? Is point-in-time availability respected?
Write	Is append, merge or interval replacement correct for retries? How many files and tasks result? Are publish boundaries complete?
Delta	What version committed? Did the watermark advance afterwards? Can a retry recognize or safely reproduce the result?
Maintenance	Are compaction, clustering and statistics justified? Is retention safe? Are automatic and manual jobs duplicating work?
Compute	Are workers busy? Is there enough task parallelism? Did Photon accelerate supported work? Are configuration overrides actually effective?
Decision	Did the same correct output become faster or cheaper? What did one change improve? What regression or rollback condition is recorded?

29 One page concept map



Responsibilities across storage table state and execution

Read the map as responsibilities, not as a rule that every query executes every branch. ADLS stores objects. Parquet organizes values inside those objects. Delta defines the table snapshot. Spark plans and distributes work. The supported engine operators process each task's data. The writer creates new files, and a transaction publishes them.

Three boundaries to remember: storage layout is not task distribution; a Parquet footer is not the Delta log; aggregate task time is not query wall-clock time.

30 Video production package

This package specifies a 14 minute teaching video using the same rows and diagrams. The duration includes spoken narration, cursor demonstrations, animation and deliberate pauses for reading. Narration is written exactly below; timing should be adjusted slightly during recording for the speaker's natural pace. No rendered MP4 or recorded narration is included.

Production settings: 1920 by 1080, 30 fps, large high-contrast labels, captions matching the narration, and no more than six visible rows at once during movement. Use the full twelve-row table only as a paused reference. Keep the colour for each key consistent: A-S1 blue, B-S1 orange, A-S2 green, B-S2 purple. Display R labels separately from business keys.

Use the supplied document diagrams as composition references. Rebuild moving elements as editable shapes in the chosen video editor; do not zoom a whole portrait page into an unreadably small video frame. Retain a visible date filter, version number or grouping key whenever it explains the movement. Use simple fades for skipped data and arrows only for actual movement.

Scene 1 from 00 00 to 01 00

On screen: the twelve source rows, then the ten-row daily result. Highlight R01 and R02, then replace their quantities 2 and 3 with one daily quantity 5. Animation: 0–10 seconds title; 10–35 seconds row merge; 35–50 seconds grain labels; 50–60 seconds pause on total 35.

Narration: “Let us follow one small set of sales rows all the way through the pipeline. These two rows are the same item, store and date. One has quantity two and the other has quantity three. Our forecasting data product needs one daily row, so the result is five. We will keep following these same rows. Later, we will scale the idea to a terabyte. The first check is always the meaning of a row. If that is unclear, a faster pipeline can still give the wrong answer.”

Scene 2 from 01 00 to 02 00

On screen: ADLS table location, three files and `_delta_log`. Expand one file into three row groups. Animation: 0–20 seconds physical objects; 20–40 seconds nested row groups; 40–60 seconds show task labels separately.

Narration: “The table is stored as several files. One terabyte does not mean one enormous file, and it does not mean the whole table is loaded into one machine. Each Parquet file can contain several row groups. Later, Spark creates pieces of work called partitions. Those are not the same as the files or the storage directories. Keep those ideas separate. Storage tells us where the data lives. The execution plan tells us how the work will be divided.”

Scene 3 from 02 00 to 03 15

On screen: row group 0 as rows, then date, item, store and quantity column chunks. Reveal pages, then footer. Animation: 0–25 seconds transpose the display; 25–45 seconds repeated S1 values become dictionary codes; 45–65 seconds show footer location; 65–75 seconds reading pause.

Narration: “Inside a row group, Parquet keeps the values from each column together. Our store column repeats S1, so the writer can represent it compactly. The quantity values are stored in their own column chunk. Those chunks contain pages of encoded data. At the end of the file is metadata called the footer. It describes the schema, row groups and column locations. It can also contain statistics such as minimum, maximum and null count. The footer is a map to the data. It is not another copy of the rows.”

Scene 4 from 03 15 to 04 30

On screen: actual extracted footer statistics. Apply September 2 predicate. Dim groups 0 and 2; retain R05–R08. Then show the seven-day large-table query selecting three columns. Animation: 0–35 seconds date exclusion; 35–55 seconds column selection; 55–75 seconds mixed-date counterexample.

Narration: “Now we ask for September second. The first row group contains September first, and the last contains September third. Their date ranges cannot match, so a compatible reader can skip them. It reads the middle group

and checks the rows that remain. If we need only three columns, it can also avoid the unrelated column chunks. But watch what happens when every row group contains mixed dates. The date ranges now overlap the filter. We still return the same answer, but we may have to read much more. A selective result does not guarantee a selective scan.”

Scene 5 from 04 30 to 05 40

On screen: version 1 active files F0 F1 F2; new F0b in a pending area; version 2 swaps F0 for F0b. Animation: 0–20 seconds old snapshot; 20–40 seconds write and failure alternative; 40–60 seconds atomic publication; 60–70 seconds old/new totals.

Narration: “Delta decides which files belong to the committed table. Here version one includes F0, F1 and F2. We correct one quantity and write a replacement file, F0b. While that file is being written, readers continue to use the old snapshot. When the transaction commits, the new snapshot replaces F0 with F0b. That is why readers do not see half a completed replacement. If the write fails before commit, the new file is not part of the table. The old physical file may remain for history until it is eligible for cleanup.”

Scene 6 from 05 40 to 06 50

On screen: driver above three executors; six task slots; read-filter-project operations fused within tasks. Animation: 0–20 seconds lazy plan; 20–30 seconds write action; 30–55 seconds task waves; 55–70 seconds stage boundary.

Narration: “Our DataFrame code describes the work. The driver plans it and coordinates execution. The executor tasks process partitions of the data. A simple filter can run independently within each partition. The main distributed computation starts when an action needs the result. In this example, writing the output is that action. A stage contains tasks that can perform their work before the next required shuffle boundary. One notebook cell is not necessarily one job, and one executor can run many tasks over time.”

Scene 7 from 06 50 to 08 10

On screen: R01 and R03 on E1, R02 on E2, R04 on E3. Move A partial quantities to bucket 0 and B partial quantities to bucket 1. Animation: 0–20 seconds placement; 20–45 seconds local sums; 45–65 seconds network movement; 65–80 seconds final totals.

Narration: “R01 and R02 have the same daily key, but different tasks read them. To calculate the complete total, their values have to meet. Spark can first calculate partial sums locally. Then the shuffle sends the relevant pieces to the same destination. Two and three become five. Four and one also become five for the other item. This movement has a cost: intermediate storage, network transfer and downstream reading. We should reduce unnecessary data before it moves, but we cannot simply remove the need to bring matching keys together.”

Scene 8 from 08 10 to 09 20

On screen: two-row item lookup copied to workers, followed by duplicate A counterexample. Animation: 0–25 seconds broadcast; 25–45 seconds local matches; 45–60 seconds duplicate lookup; 60–70 seconds total changes from 35 to 56.

Narration: “For a genuinely small lookup, we can copy the lookup to the workers and match sales locally. This is a broadcast join. The large sales table does not need to move for that join. But size is only part of the decision. If the lookup has two matching rows for A, each A sale appears twice. In our sample, the quantity total jumps from thirty-five to fifty-six. Broadcasting that lookup would only make the wrong answer faster. Check the business relationship first, then choose the execution strategy.”

Scene 9 from 09 20 to 10 25

On screen: task duration bars with one 30-minute tail, then valid salted SUM and final recombination. Animation: 0–20 seconds imbalance; 20–45 seconds split a hot key; 45–65 seconds recombine and validate total.

Narration: “A stage can finish slowly because one task has much more work than the others. First check whether the hot key is valid. It might be a missing-key default or a join problem. Adaptive execution can help with some

supported skewed joins. For a valid sum, a carefully designed two-stage calculation can sometimes divide a hot key and combine the partial totals afterwards. More partitions alone do not automatically split one key. Any mitigation must preserve the original business result.”

Scene 10 from 10 25 to 11 35

On screen: daily rows enter two writer tasks; pages and footer complete; Delta commit publishes D0 and D1. Then show small-file compaction and retention cleanup as separate actions. Animation: 0–35 seconds write; 35–50 seconds commit; 50–70 seconds maintenance distinction.

Narration: “Writer tasks turn their output into Parquet files. They encode columns, build row groups and finish the footer. Delta then publishes the successful files in a committed table version. Too many tiny files can make future reads expensive. Too few output tasks can reduce parallelism. Compaction repackages active data into more suitable files. Vacuum does something different: it removes eligible obsolete physical files. It does not sort the active data or calculate better features. Keep the write, layout and retention decisions separate.”

Scene 11 from 11 35 to 12 40

On screen: identical query under standard Spark SQL and Photon, with supported operators highlighted. Keep shuffle arrows on both sides. Animation: 0–25 seconds common plan; 25–45 seconds native operators; 45–65 seconds Python boundary and cost comparison.

Narration: “Photon changes how supported operations execute. It does not change the intended answer, the Delta table or the need for required data movement. A Python notebook can benefit when it uses built-in DataFrame operations, because those operations are planned by the engine. Arbitrary Python functions are different. I would benchmark the same input and compare elapsed time, total cost and correctness. A faster run is valuable, but it is not automatically cheaper, and it does not excuse an unnecessary full-table scan.”

Scene 12 from 12 40 to 14 00

On screen: redraw the user's aggregate section with 778.33 M before shuffle and 17.21 M after final grouping. Show a separate wall-clock label and summed-worker-time label. End on read → join → aggregate → write checklist as four cards. Animation: 0–25 seconds row path; 25–50 seconds timing distinction; 50–70 seconds investigation order; 70–80 seconds final pause.

Narration: “In this real profile, there are still around seven hundred and seventy-eight million rows going into an aggregation shuffle, and around seventeen million after the final aggregate. That is an area to investigate, not yet a proven diagnosis. I would check the grouping keys, mapping grain, bytes moved, spill and task balance. I would also confirm what the timing labels mean. Combined task time across many cores is not the same as elapsed query time. My approach is to follow the data, preserve the correct grain, reduce unnecessary work, and prove each improvement with measurements.”

31 Primary references and coverage audit

References are official specifications or product documentation checked on 23 September 2026. Example data, diagrams and benchmarks are original teaching examples. Use the deployed runtime documentation for implementation-specific behavior.

[1] Apache Parquet file format. <https://parquet.apache.org/docs/file-format/>

[2] Apache Spark SQL performance tuning, versioned baseline. <https://spark.apache.org/docs/3.5.5/sql-performance-tuning.html>

[3] Apache Parquet metadata. <https://parquet.apache.org/docs/file-format/metadata/>

[4] Apache Parquet encodings. <https://parquet.apache.org/docs/file-format/data-pages/encodings/>

[5] Apache Parquet page index specification.
<https://github.com/apache/parquet-format/blob/master/PageIndex.md>

[6] Apache Spark Parquet data source. <https://spark.apache.org/docs/latest/sql-data-sources-parquet.html>

[7] Apache Spark cluster overview. <https://spark.apache.org/docs/latest/cluster-overview.html>

[8] Delta Lake transaction protocol. <https://github.com/delta-io/delta/blob/master/PROTOCOL.md>

[9] Delta Lake concurrency control. <https://docs.delta.io/concurrency-control/>

[10] Azure Databricks row-level concurrency.
<https://learn.microsoft.com/en-us/azure/databricks/optimizations/isolation/row-level-concurrency>

[11] Delta Lake utility commands and retention. <https://docs.delta.io/delta-utility/>

[12] Azure Databricks deletion vectors. <https://learn.microsoft.com/en-us/azure/databricks/tables/features/deletion-vectors>

[13] Azure Databricks adaptive query execution.
<https://learn.microsoft.com/en-us/azure/databricks/optimizations/ake>

[14] Azure Databricks data file sizing. <https://learn.microsoft.com/en-us/azure/databricks/tables/tune-file-size>

[15] Azure Databricks selective overwrite. <https://learn.microsoft.com/en-us/azure/databricks/delta/selective-overwrite>

[16] Azure Databricks liquid clustering. <https://learn.microsoft.com/en-us/azure/databricks/tables/clustering>

[17] Azure Databricks predictive optimization.
<https://learn.microsoft.com/en-us/azure/databricks/optimizations/predictive-optimization>

[18] Azure Databricks Photon. <https://learn.microsoft.com/en-us/azure/databricks/compute/photon>

[19] Azure Databricks query profile. <https://learn.microsoft.com/en-us/azure/databricks/sql/user/queries/query-profile>

[20] Azure Databricks data skipping. <https://learn.microsoft.com/en-us/azure/databricks/delta/data-skipping>

Requested coverage	Where to find it
One continuous sample and 1 TB scaling	Chapters 1 and 2, reused throughout
Rows, row groups, chunks, pages, footer and statistics	Chapters 3 to 5 and real metadata lab
Reader, execution engine and writer responsibilities	Chapters 6, 10, 16 and 21
Delta snapshots, commits, failures, concurrency and retention	Chapters 7 to 9
Jobs, stages, tasks, narrow and wide operations	Chapters 10 and 11
Joins, duplicate keys, skew, AQE and partition APIs	Chapters 12 to 14
Daily loads, corrections, windows and configured backfills	Chapters 15 and 17
Read, transform, write, maintenance and compute diagnosis	Chapters 16 to 20
Spark versus Photon and realistic cost comparison	Chapters 21 and 25
Both supplied photographs and careful interpretation	Chapters 22 to 24
Complete illustrative troubleshooting case	Chapter 25

Requested coverage	Where to find it
Twenty interview questions and spoken explanation	Chapters 26 and 27
One-page checklist and concept map	Chapters 28 and 29
Timed video scenes, exact narration and animation instructions	Chapter 30